

MOTOR CONTROL SYSTEM

Quick Reference Manual — Rev 2.0

DR28T2.5B03-AZAKD × AZD-KEP | ROS2 Humble

Rev	Platform	Interfaces	Date
2.0	Ubuntu 22.04 + ROS2 Humble	USB Serial & EtherNet/IP	April 2026

This manual documents the 5-DOF Yuzu Peeler motor system: hardware specs, both communication interfaces (USB serial legacy and EtherNet/IP production), all ROS2 nodes, launch files, GUI, and the harvest sequence orchestrator.

System Overview — 5 Axes

Namespace	Axis	Motor	Driver / Node
motor_z	Z linear (peeler head)	DR28T	ethernet_ip_motor_node or motor_controller_node
motor_yuzu_rot	Yuzu rotation spindle	Rotary	rotary_motor_stub_node (stub)
motor_peeler_orbit	Peeler arm orbit	Rotary	rotary_motor_stub_node (stub)
motor_peeler_advance	Peeler radial advance	DR28T	ethernet_ip_motor_node or stub
motor_conveyor	Conveyor belt	Stepper	rotary_motor_stub_node (stub)

1 Hardware Specifications

1.1 Motor — DR28T2.5B03-AZAKD

Parameter	Value	Notes
Type	Compact Electric Cylinder	Ball screw, closed-loop stepper
Stroke	30 mm	Software limit set to 29 mm
Lead / Pitch	2.5 mm / rev	Used for steps_per_mm calculation
Max Speed	100 mm/s	Hard cap in node parameters
Max Thrust	20 N	
Repeat Accuracy	±0.003 mm	Ball screw shaft tip
Encoder	Absolute multi-turn	No battery required
Supply Voltage	24 / 48 VDC	Driver dependent

1.2 Driver — AZD-KEP

Parameter	Value
Series	AZ Series Closed-Loop Stepper Driver
Fieldbus	EtherNet/IP™ (CT16 conformant, Vendor ID 187)
Network Speed	10/100 Mbps (autonegotiation)
USB Port	Mini-B → MEXE02 config tool (Windows only)
Control Power	24 VDC ±5%, 0.25 A
IP Address	192.168.1.x (set via rotary switches on face)
EDS File	Download from orientalmotor.com

IP address rotary switches: IP ADDR ×16 and ×1 on the driver face. IP = 192.168.1.(16 × x16 + x1). e.g. ×16=0, ×1=2 → 192.168.1.2.
Power-cycle after changing switches.

2 Communication Interfaces

2.1 USB Serial (Legacy — motor_controller_node.py)

Motor is controlled by replaying binary MEXE02 protocol frames captured on Windows, sent over the USB Virtual COM Port (Linux: /dev/ttyACM0).

Setting	Value
Baud rate	19 200
Data bits	8
Parity	Even

Stop bits	1
Timeout	0.5 s

Position encoding: $\text{mm} \times 1000 \rightarrow \mu\text{m}$ (int32 LE). Speed encoding: $\text{mm/s} \times 1000 \rightarrow \mu\text{m/s}$ (int32 LE). XOR checksum patched per frame. Template HTML files required in working dir.

2.2 EtherNet/IP (Production — ethernet_ip_motor_node.py)

CIP explicit messaging via pycomm3. Get/Set Attribute Single on the Assembly Object (class 0x04). No implicit (UDP) connection required — each command is a separate TCP request.

Direction	Instance	Attr	Size	Purpose
Scanner → Driver (cmd)	101 (0x65)	3	40 B	Set Attribute Single — write motion command
Driver → Scanner (stat)	100 (0x64)	3	56 B	Get Attribute Single — read status

Output Assembly — 40 bytes (scanner → driver)

Bytes	Type	Field	Notes
0–1	H	Remote I/O (R-IN)	Set to 0 (unused)
2–3	H	Operation data number selection	Set to 0
4–5	H	Fixed I/O (IN) — control bits	See bit table below
6–7	H	Direct data op type	1 = absolute positioning
8–11	i	Direct data op position	steps (signed int32)
12–15	i	Direct data op speed	Hz = steps/s (signed int32)
16–19	i	Starting / changing rate	1 = 0.001 kHz/s (default 1 000 000)
20–23	i	Stopping deceleration	same unit (default 1 000 000)
24–25	H	Operating current	1 = 0.1%, 1000 = 100%
26–27	H	Forwarding destination	0 = execution memory
28–35	4H	Reserved / parameter R/W area	Set to 0 for motion commands
36–39	i	Write data	Set to 0 for motion commands

Python struct: `struct.Struct("<HHHH iiii HHHHHH i")` → 40 bytes

Fixed I/O (IN) bit definitions (output bytes 4–5):

Bit	Mask	Name	Description
3	0x0008	START	Execute stored-data operation
4	0x0010	ZHOME	High-speed return-to-home
5	0x0020	STOP	Deceleration stop

6	0x0040	FREE	De-energise motor coils
7	0x0080	ALM-RST	Reset active alarm (pulse ON→OFF)
8	0x0100	TRIG	Trigger direct data operation (rising edge)
9	0x0200	TRIG-MODE	0 = ON edge (default), 1 = ON level

Input Assembly — 56 bytes (driver → scanner)

Bytes	Type	Field	Notes
0–1	H	Remote I/O (R-OUT)	
2–3	H	Op data number selection_R	
4–5	H	Fixed I/O (OUT) — status bits	See bit table below
6–7	H	Present alarm code	0 = no alarm
8–11	i	Feedback position	steps (signed int32)
12–15	I	Feedback speed	Hz (unsigned int32)
16–19	i	Command position	steps (signed int32)
20–23	2H	Torque monitor / CST current	
24–27	I	Information code	
28–35	4H	Reserved / parameter R/W area	
36–55	5i	Read data + assignable monitors 0–3	

Python struct: `struct.Struct("<HHHH iIi HH I HHHH iiii")` → 56 bytes

Fixed I/O (OUT) bit definitions (input bytes 4–5):

Bit	Mask	Name	Description
1	0x0002	MOVE	Motor is currently operating
2	0x0004	IN-POS	Positioning complete
4	0x0010	HOME-END	Return-to-home complete
5	0x0020	READY	Driver ready to accept command
6	0x0040	DCMD-RDY	Ready for direct data operation (poll before TRIG)
7	0x0080	ALM-A	Alarm active (normally open)
8	0x0100	TRIG_R	TRIG acknowledged by driver
10	0x0400	SET-ERR	Error in direct data settings
11	0x0800	EXE-ERR	Direct data execution failed

Direct Data Operation — TRIG ON-Edge Sequence (manual §6-3)

Step	Action
1	Poll input assembly until DCMD-RDY (bit 6 of FIO-OUT) is ON.
2	Write output assembly: set op type=1, position, speed, accel, decel. TRIG = 0.
3	Wait 1 ms, then write output again with TRIG = 1 (rising edge triggers move).
4	Wait 5 ms, write output with TRIG = 0 (clear after driver latches).
5	Poll input until IN-POS (bit 2) or HOME-END (bit 4) → publish status='idle'.
6	If ALM-A (bit 7), SET-ERR (bit 10), or EXE-ERR (bit 11) → publish status='error'.

Unit Conversions ($\text{steps_per_mm} = \text{encoder_pulses_per_rev} / \text{screw_lead_mm_per_rev}$)

ROS parameter	Assembly unit	Conversion formula
pos_mm (mm)	position (steps)	$\text{steps} = \text{round}(\text{pos_mm} \times \text{steps_per_mm})$
spd_mms (mm/s)	speed (Hz)	$\text{hz} = \text{round}(\text{spd_mms} \times \text{steps_per_mm})$
acceleration_mms2 (mm/s ²)	accel_raw (1=0.001 kHz/s)	$\text{raw} = \text{round}(\text{accel_mms2} \times \text{steps_per_mm})$ or 1 000 000 if 0

Example: DR28T with 1000 pulses/rev and 2.5 mm/rev lead → steps_per_mm = 400. Moving 10 mm at 5 mm/s → position=4000 steps, speed=2000 Hz.

3 ROS2 Nodes

3.1 Topic API (identical for both driver nodes)

Topic (relative)	Type	Dir	Description
~/motor_cmd	std_msgs/Float32MultiArray	Sub	data[0]=pos (mm), data[1]=speed (mm/s)
~/motor_home	std_msgs/Empty	Sub	Trigger return-to-home
~/motor_status	std_msgs/String	Pub	idle busy running error

3.2 motor_controller_node.py — USB Serial (Legacy)

Parameter	Default	Description
com_port	/dev/ttyACM0	Serial device path
baudrate	19200	Must match MEXE02 setting
max_position_mm	29.0	Soft upper limit (mm)
max_speed_mms	100.0	Hard speed cap (mm/s)
frame_delay	0.1	Inter-frame delay (s)
heartbeat_interval	0.2	Watchdog ping period (s)

3.3 ethernet_ip_motor_node.py — EtherNet/IP (Production)

Requires: `pip install pycomm3`

Parameter	Default	Description
ip_address	192.168.1.2	Driver IP (set via rotary switches)
motor_id	"motor"	Label for log messages
steps_per_mm	100.0	!! CALIBRATE !! — encoder_pulses_per_rev / screw_lead_mm_per_rev
max_position_mm	29.0	Soft upper limit (mm)
min_speed_mms	0.1	Minimum commanded speed (mm/s)
max_speed_mms	100.0	Maximum commanded speed (mm/s)
acceleration_mms2	0.0	0 = use driver default (1000 kHz/s)
deceleration_mms2	0.0	0 = use driver default (1000 kHz/s)
poll_interval_s	0.05	Status polling period (s)
dcmd_rdy_timeout_s	3.0	Max wait for DCMD-RDY before move

Tune steps_per_mm at runtime without rebuilding:

```
ros2 param set /motor_z/ethernet_ip_motor_node steps_per_mm 400.0
```

3.4 rotary_motor_stub_node.py — Simulation Stub

Simulates rotary and conveyor motors so the full ROS2 graph can run without hardware. Publishes idle/spinning/stepping status after configurable delays.

Topic (relative)	Type	Dir	Description
~/motor_spin	std_msgs/Float32	Sub	Set target RPM
~/motor_stop	std_msgs/Empty	Sub	Stop rotation
~/motor_step	std_msgs/Empty	Sub	Advance one conveyor step
~/motor_status	std_msgs/String	Pub	idle spinning stepping

3.5 harvest_sequence_node.py — Harvest Orchestrator

Coordinates all 5 axes through one yuzu peel cycle. Receives optional yuzu measurements from the image team and adapts Z depth and blade advance accordingly.

Topic	Type	Dir	Description
/yuzu_detected	std_msgs/Float32MultiArray	Sub	[x_off, y_off, long_mm, short_mm] — optional, improves depth
/harvest/start	std_msgs/Empty	Sub	Start one peel cycle
/harvest/abort	std_msgs/Empty	Sub	Emergency stop all motors
/harvest/status	std_msgs/String	Pub	idle feeding lowering spinning_up peeling retracting spinning_down raising done error aborted

Key harvest parameters (all tunable via ros2 param set):

Parameter	Default	Description
yuzu_rotation_rpm	225.0	Yuzu spindle speed (200–250 rpm spec)
peeler_orbit_rpm	25.0	Peeler orbit speed (20–30 rpm spec)
peel_duration_s	1.5	Peel arc time (0.5 rev at 20 rpm)
z_lower_pos_mm	25.0	Z descent onto yuzu (scales with long_axis_mm)
peeler_advance_mm	10.0	Blade push depth (scales with short_axis_mm)
conveyor_settle_s	1.2	Wait after conveyor step

4 Launch Files

Two separate launch files — use the one that matches your hardware connection:

File	Command	Linear driver used
motor_usb.launch.py	ros2 launch motor_controller motor_usb.launch.py	motor_controller_node (USB serial)
motor_lan.launch.py	ros2 launch motor_controller motor_lan.launch.py	ethernet_ip_motor_node (EtherNet/IP)

Both files launch the rotary stubs and harvest_sequence_node automatically. Stubs are skipped for any namespace that already has a real driver configured.

4.1 motor_usb.launch.py — configure SERIAL_MOTORS list

```
SERIAL_MOTORS = [ { "ns": "motor_z", "motor_id": "motor_z", "com_port": "/dev/ttyACM0" }, ]
```

4.2 motor_lan.launch.py — configure ETHERNET_IP_MOTORS list

```
ETHERNET_IP_MOTORS = [ { "ns": "motor_z", "motor_id": "motor_z", "ip": "192.168.1.2",  
"steps_per_mm": 400.0 }, ]
```

Build and launch:

```
cd ~/ros2_ws  
colcon build --packages-select motor_controller  
source install/setup.bash  
# USB method:  
ros2 launch motor_controller motor_usb.launch.py  
# LAN method:  
ros2 launch motor_controller motor_lan.launch.py
```

5 GUI — motor_gui.py

Run from the motion_cmd directory:

```
python motor_controller/motor_gui.py
```

Tabbed Tkinter GUI. Each motor has its own tab; a Harvest tab triggers and monitors the full peel cycle.

Element	Function
Per-motor tab	Status dot, position/speed inputs, Send Command, Go Home, log box
Status dot	Colour-coded live /motor_status feed
Position entry	0 – 29 mm; validated before publish
Speed entry	0.1 – 100.0 mm/s; validated before publish
Send Command	Publishes Float32MultiArray to ~/motor_cmd
Go Home	Confirmation dialog → publishes to ~/motor_home
Harvest tab — Start	Publishes Empty to /harvest/start

Harvest tab — Abort	Publishes Empty to /harvest/abort (emergency)
Harvest status bar	Shows /harvest/status live

Colour	Status	Meaning
● Green	idle	Ready, no active motion
● Orange	busy	Command being transmitted
● Blue	running	Motor in motion
● Red	error	Command rejected or alarm

Requires: rclpy, tkinter, std_msgs. ROS2 spin runs in a daemon thread — closing the window shuts down cleanly.

6 Safety & Operating Limits

Limit	Value	Enforced By
Max position	29.0 mm	Node parameter + GUI validation
Min position	0.0 mm	Node + GUI
Max speed	100.0 mm/s	Node parameter + GUI validation
Min speed	0.1 mm/s	Node + GUI
Physical stroke	30 mm	Hardware — do not override max_position_mm above 29

Warnings

- ▲ Always home the motor after power-on before sending position commands.
- ▲ Do NOT set max_position_mm above 29.0 mm — physical stroke is 30 mm; over-travel damages the cylinder.
- ▲ USB: ensure /dev/ttyACM0 is accessible: `sudo chmod 666 /dev/ttyACM0`
- ▲ USB: close MEXE02 on Windows before running the ROS2 node — only one process can hold the COM port.
- ▲ LAN: calibrate steps_per_mm before the first move, or the motor will travel the wrong distance.
- ▲ LAN: set PC NIC to 192.168.1.x subnet before launch, otherwise connection will fail silently.
- ▲ Harvest abort (/harvest/abort) homes Z and advance axes immediately — ensure clear area before pressing.

7 Troubleshooting

7.1 USB Serial Issues

Symptom	Likely Cause	Fix
Failed to open /dev/ttyACM0	Permission denied or cable unplugged	<code>sudo chmod 666 /dev/ttyACM0</code>
No motor response after frames sent	Wrong baud/parity or motor not powered	Confirm 24 V supply; check serial settings
FileNotFoundError: write25mm.html	Node launched from wrong directory	Run from motion_cmd directory
Motor stops mid-motion	Heartbeat thread died (SerialException)	Restart node; check USB cable

7.2 EtherNet/IP (LAN) Issues

Symptom	Likely Cause	Fix
EtherNet/IP connect failed	Wrong IP / subnet mismatch / cable unplugged	Ping driver IP; set PC NIC to 192.168.1.x
ModuleNotFoundError: pycomm3	pycomm3 not installed	<code>pip install pycomm3</code>

Timeout waiting for DCMD-RDY	Driver not ready or alarm active	Check PWR/ALM LED; clear alarm via ALM-RST
Motor moves wrong distance	steps_per_mm not calibrated	Measure actual travel; adjust steps_per_mm
SET-ERR after TRIG	Op type / position out of range	Check position limits; confirm op type = 1
EXE-ERR after TRIG	Driver not in executable state	Send ZHOME first; check driver enable signal
NS LED red/blinking	EtherNet/IP communication error	Check cable; verify IP address on driver
PWR/ALM LED blinking red	Driver alarm active	Read alarm code from /harvest/status or GUI; clear with ALM-RST

7.3 General ROS2 Issues

Symptom	Fix
ModuleNotFoundError: rclpy	source /opt/ros/humble/setup.bash
Node not found after colcon build	source install/setup.bash && rebuild
GUI shows no status updates	Confirm motor driver node is running: ros2 node list
Position rejected (out of range)	Send value between 0.0 and 29.0 mm